

Documentación Villian's Tale

Eloy Martín, Daniel de la Cruz y Abel Anneix



Introducción	2
Carga de trabajo	2
Detalles técnicos	2
Lenguaje RPG Maker	2
Base de datos	2
Estructura de ficheros	2
Guardado	5
.RPGSAVE	5
SuperToolsEngine	5
HUDMaker	8
Gestión y Opciones del Editor	9
Mundo	10
Jugabilidad	11
Batallas	11
3. Habilidades a aprender (Centro Inferior)	12
4. Atributos (Derecha Superior)	12
Configuración General e Invocación	13
Mensaje y Armas requeridas	13
Daño y Efectos	14
Notas	14
Buffs y debuffs	14
Configuración General	15
Condiciones de Eliminación	15
Mensajes	15
Atributos	15
1. Objetos (Consumibles y Elementos Clave)	16
2. Armas	17
3. Armaduras	18
Comercio	19
Temporalización	20
Fase primera	20
Fase Segunda: Sistema de Combate y Dinámicas de Batalla	22
Desarrollo y Tareas Clave:	22
Fase Tercera: Implementación de Infraestructura y Contenido Base	22
Desarrollo y Tareas Clave:	23
Fase Cuarta: Persistencia de Datos y Atmósfera Sonora	23
Desarrollo y Tareas Clave:	23
Fase Quinta y Final: Documentación, Cierre y Presentación	24
Desarrollo y Tareas Clave:	24

Introducción

El juego se desarrolla en un mundo medieval en el que nos posicionamos en la piel del villano de la historia. El objetivo principal es acabar con los cuatro héroes que nos atormentan y derrocan al principio del juego. En la presente demo nos centraremos hasta el combate con el primer jefe, que es, ni más ni menos, que uno de los cuatro héroes y antagonistas principales.

Carga de trabajo

Se ha dividido la carga de trabajo para las presentaciones quincenales en proporciones aparentemente iguales y se han intercalado trabajos para la equidad interpersonal. Dichas divisiones a groso modo son:

- **Scripts (Alumno 1):** Desarrollado por *Eloy Martín*. Cubre el apartado técnico en cuanto al desarrollo de scripts y subtareas dentro del entorno.
- **Diseño (Alumno 2):** Desarrollado por *Daniel de la Cruz*. Cubre el desarrollo de mapas, niveles e interfaces, cumpliendo el frontend, junto a tareas derivadas de las mismas.
- **Datos (Alumno 3):** Desarrollado por *Abel Anneix*. Cubre el desarrollo y mantenimiento de la base de datos junto a sus tareas dependientes.

Detalles técnicos

La demo, al más puro *estilo RPG* (Role Play Game), ha sido **desarrollada con RPG Maker**, un programa que facilita el desarrollo de este estilo de juegos. Este programa también *permite modificaciones como plugins*.

Lenguaje RPG Maker

El programa *interpreta código JavaScript* para el desarrollo del juego y sus eventos. Lo que permite gran personalización en el entorno.

Base de datos

La base de datos está *incorporada* con un óptimo funcionamiento que permite añadir y modificar las habilidades, armas y estadísticas del juego a merced sin necesidad de instalar nada más que, evidentemente, el propio RPG Maker. Aún así, también **permite las modificaciones en las tablas** que sean necesarias. La base de datos se almacena en *formato JSON*.

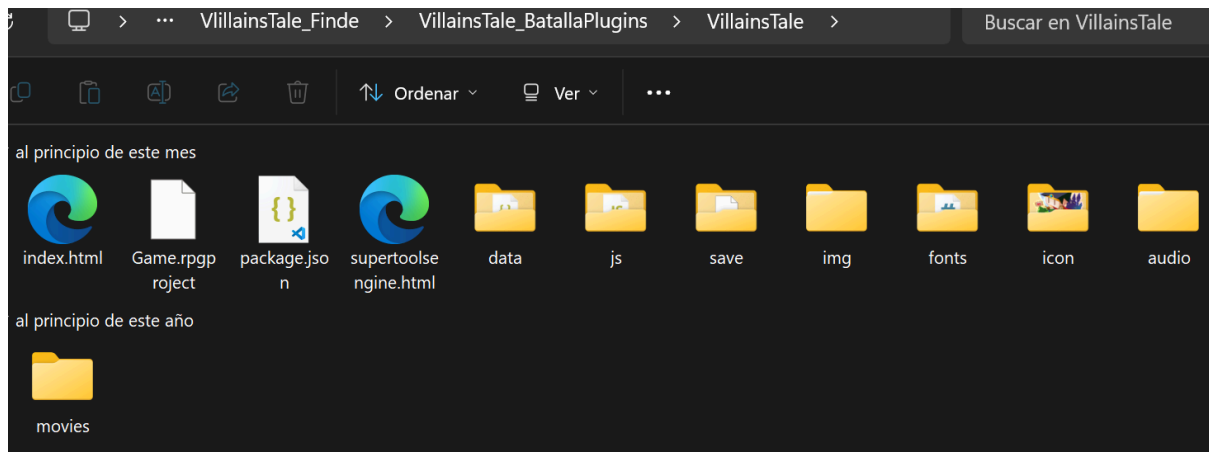
Estructura de ficheros

El programa estructura los ficheros de manera jerárquica y ordenada, y esto lo hace automáticamente. He aquí una ilustración de la estructura en cuestión:

```

C:\.
├── locales
├── swiftshader
├── www
│   ├── audio
│   │   ├── bgm
│   │   ├── bgs
│   │   ├── me
│   │   └── se
│   ├── data
│   ├── fonts
│   ├── icon
│   ├── img
│   │   ├── animations
│   │   ├── battlebacks1
│   │   ├── battlebacks2
│   │   ├── characters
│   │   ├── enemies
│   │   ├── faces
│   │   ├── parallaxes
│   │   ├── SumRndmDde
│   │   │   └── hud
│   │   │       ├── gauge_backs
│   │   │       ├── gauge_images
│   │   │       ├── numbers
│   │   │       └── pictures
│   │   ├── sv_actors
│   │   ├── sv_enemies
│   │   ├── system
│   │   ├── tilesets
│   │   ├── titles1
│   │   └── titles2
│   ├── js
│   │   ├── libs
│   │   └── plugins

```



- **index.html**: Contiene los *scripts JS* para cargar el juego y el *estilaje CSS*.
- **Game.rpgproject**: Este es el *ejecutable* que interpreta RPG Maker para abrir el juego como archivo. Luego al compilarlo como ejecutable dicho fichero sería *Game.exe*.
- **package.json**: Es el fichero de datos que almacena el *nombre del proyecto*, define el *fichero index.html* que tiene que cargar y también qué *ventana debe cargar al iniciar*.
- **data/**: Es el directorio que almacena lo relevante a la *base de datos*. Todas las tablas en formato JSON.
- **js/**: Contiene todo lo pertinente a la *lógica del juego* y los scripts inherentes a ella.
- **img/**: Es el directorio que almacena las *imágenes*.
- **audio/**: Es el directorio que almacena los *audios*.
- **save/**: En este directorio se almacenan los *ficheros de guardado* (rpgsave).
- **fonts/**: Fuentes en formatos o *.woff* o *.ttf*.
- **icon/**: Almacena *iconos*.
- **movies/**: Almacena *ficheros de vídeo* para introducciones o cinemáticas.

Cabe añadir que las subcarpetas no se han recogido para no alargar innecesariamente la introducción. Pero contienen información de cualidades más específicas.

Guardado

El *guardado es manual*. Una vez iniciado el juego en el menú de pausa existe la opción de guardado. Una vez guardada cierta partida se almacena en el directorio “saves” del juego un fichero “.rpgsave”.

.RPGSAVE

Los *ficheros “.rpgsave”* son objetos de JavaScript serializados en formato **JSON** y posteriormente codificados en **base64**, dando como resultado estos ficheros reversibles para la deserialización posterior del propio juego. Estos ficheros *almacenan toda la información de la partida* para poder ser cargada en cualquier momento. Entre esta información están:

- Lista de personajes en el equipo
- La vida de cada personaje en el equipo.
- El inventario de cada personaje en el equipo.
- El nivel del de cada personaje en el equipo.
- La experiencia de cada personaje en el equipo.
- Contador de cuántas veces se ha sobrescrito el fichero, es decir, cuántas veces se ha guardado esa partida en ese fichero.
- Ubicación del jugador.
- Coordenadas del jugador.
- Orientación del jugador.
- Etcétera.

Plugins instalados

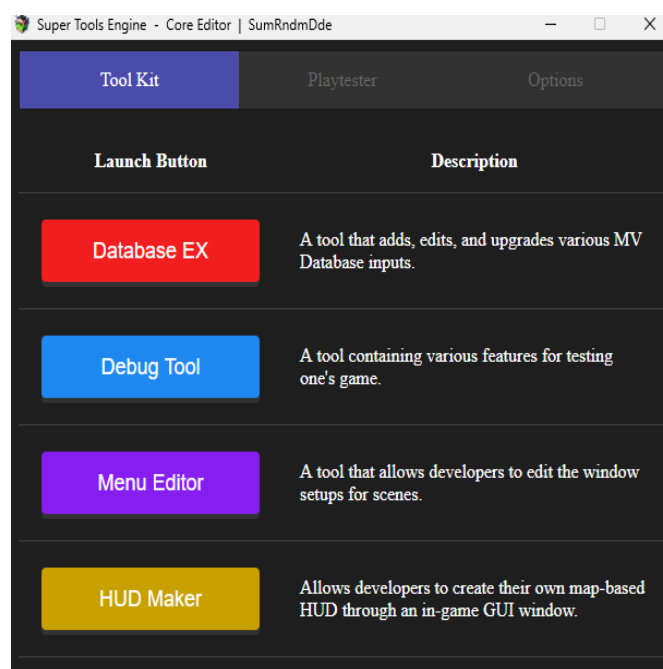
Los **plugins en RPG Maker** son *extensiones descargables que facilitan el desarrollo* del juego. Además, *permiten la parametrización* de los mismos para la personalización de la experiencia. Estos también se pueden activar y desactivar. En este proyecto hemos implementado **dos** plugins.

SuperToolsEngine

Añade un **menú** para el “**meta testing**” dentro del entorno jugable. Añade opciones como:

Tool Kit:

Botón de Lanzamiento	Función Principal	Descripción de Uso
Database EX (Rojo)	Editor de Base de Datos	Permite añadir, editar y actualizar de forma avanzada los diferentes datos y entradas de la base de datos de RPG Maker MV.
Debug Tool (Azul)	Herramienta de Depuración	Contiene varias funciones y características diseñadas específicamente para probar el juego y detectar errores durante el desarrollo.
Menu Editor (Morado)	Editor de Menús	Permite a los desarrolladores modificar y personalizar la configuración y disposición de las ventanas en las distintas escenas del juego.
HUD Maker (Amarillo)	Creador de Interfaces (HUD)	Permite diseñar una interfaz gráfica personalizada (como barras de vida, mapas o indicadores) directamente sobre el mapa mediante una ventana GUI dentro del juego.



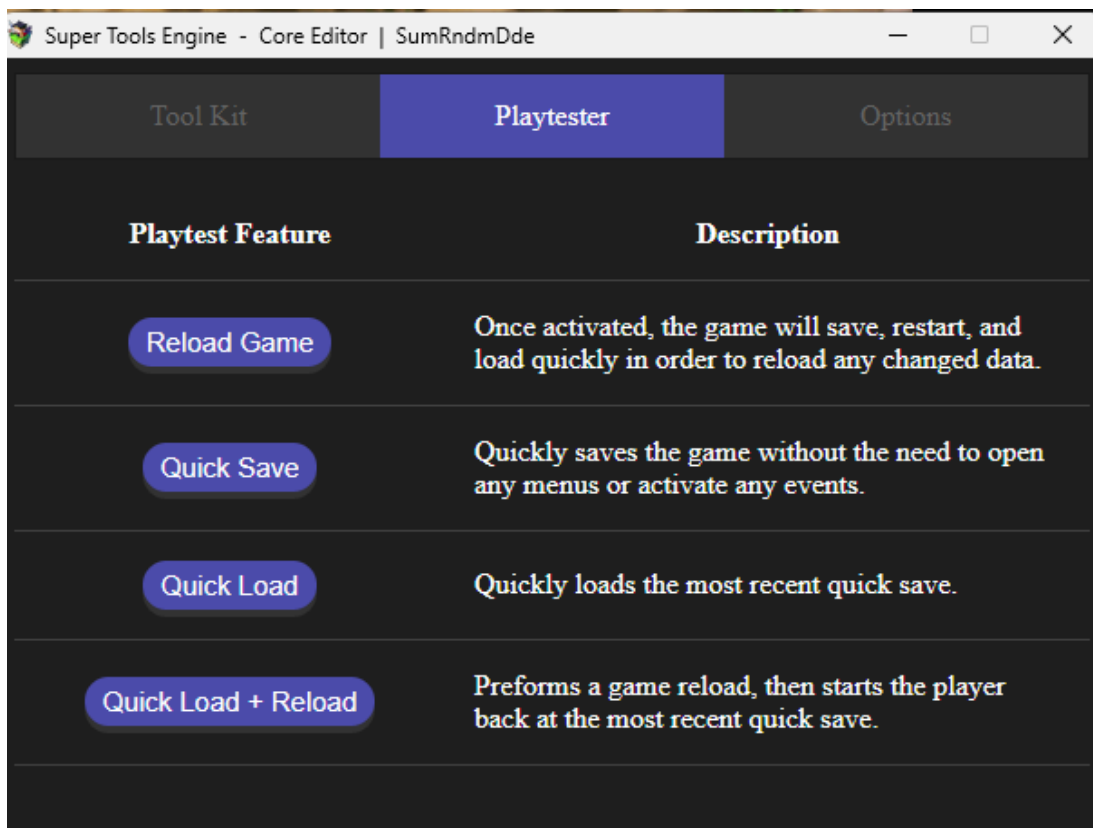
PlayTester:

Reload Game (Recargar Juego): Al activarlo, el juego guarda la partida automáticamente, se reinicia y se carga de forma rápida. Sirve para aplicar inmediatamente cualquier cambio que hayas hecho en los datos o archivos del juego sin tener que cerrarlo por completo.

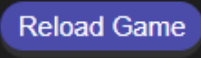
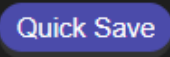
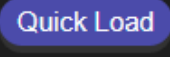
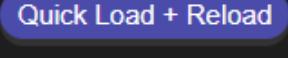
Quick Save (Guardado Rápido): Guarda la partida al instante con un solo clic, sin necesidad de abrir el menú del juego ni activar ningún evento de guardado.

Quick Load (Carga Rápida): Carga rápidamente la partida guardada con la función de *Quick Save* más reciente.

Quick Load + Reload (Carga y Recarga Rápida): Realiza una recarga completa del juego (actualizando los datos modificados) y, acto seguido, sitúa al jugador directamente en el punto del último *Quick Save*.



The screenshot shows a software window titled "Super Tools Engine - Core Editor | SumRndmDde". It features a dark-themed interface with a top navigation bar containing three tabs: "Tool Kit", "Playtester" (which is selected and highlighted in blue), and "Options". Below the tabs is a table with two columns: "Playtest Feature" and "Description". The table lists four features, each with a blue button icon in the first column and a text description in the second column.

Playtest Feature	Description
	Once activated, the game will save, restart, and load quickly in order to reload any changed data.
	Quickly saves the game without the need to open any menus or activate any events.
	Quickly loads the most recent quick save.
	Preforms a game reload, then starts the player back at the most recent quick save.

HUDMaker

Permite una **personalización más desarrollada** y profunda para las **interfaces**. El propósito en *Villian's Tale* es la interfaz de *pelea modificada* en los combates. Permite establecer variables para visualizar los “puntos de vida” y los “puntos de magia” de los destinos personajes del equipo, así como customizar sus iconos, mostrar sus nombres, etc. . .

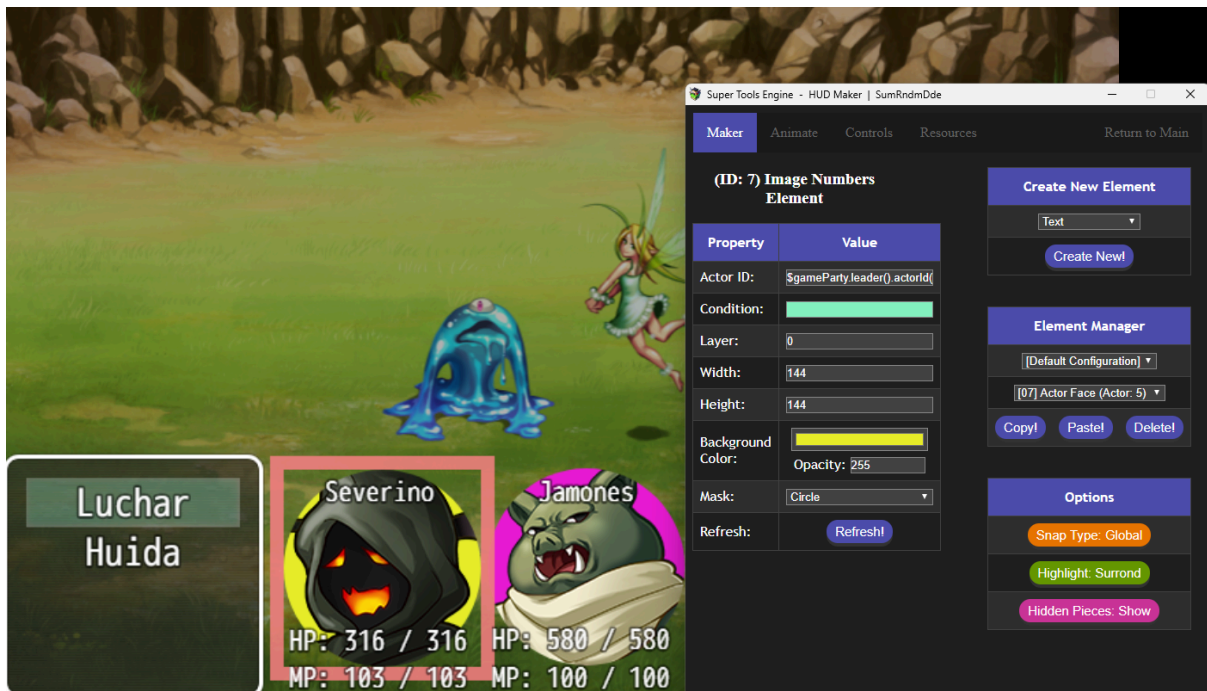
Muestra los parámetros del elemento actualmente seleccionado (en este caso, un elemento de imagen asignado a la cara del personaje principal):

- **Actor ID:** Define a qué personaje pertenece el elemento. Usa código (`$gameParty.leader().actorId()`) para que detecte automáticamente al líder del grupo.
- **Condition:** Condición lógica (en código) para que el elemento se muestre o se oculte en pantalla.
- **Layer:** La capa de renderizado (capa 0). Determina qué elementos se dibujan por encima o por debajo de otros.
- **Width / Height:** Ancho y alto del elemento en píxeles (144×144).
- **Background Color / Opacity:** Color de fondo (amarillo) y su nivel de opacidad (255, totalmente opaco).
- **Mask:** Aplica una máscara de forma al elemento (configurado en "Circle" para que la imagen se vea redonda).
- **Refresh:** Botón para actualizar y aplicar los cambios visuales

[illegible]

Gestión y Opciones del Editor

- **Create New Element (Crear Nuevo Elemento):** Permite elegir un tipo de componente (como "Text", imágenes, barras de estado, etc.) desde el menú desplegable y crearlo con el botón *Create New!*.
- **Element Manager (Gestor de Elementos):** * Permite seleccionar qué elemento de la interfaz estás editando (actualmente el elemento [07] Actor Face).
 - Incluye botones rápidos para **Copiar (Copy!)**, **Pegar (Paste!)** o **Borrar (Delete!)** dicho componente.
- **Options (Opciones de Ajuste):**
 - **Snap Type (Global):** Configura el tipo de alineación o cuadrícula magnética para mover los elementos con precisión.
 - **Highlight (Surround):** Resalta el elemento seleccionado con un borde o contorno para saber qué estás editando en pantalla.
 - **Hidden Pieces (Show):** Permite elegir si se muestran u ocultan en el editor aquellos elementos que actualmente tienen su condición en "falso" (ocultos en el juego)



Mundo

El mundo se divide en *diversas secciones explorables*; tanto en interiores como exteriores. Está *cargado de descripciones* hilarantes en determinados elementos del escenario, como estatuas, bloques de oro y personajes interactivos.

Como el mundo se desarrolla en un **escenario medieval** se hace uso de *colores vivos para exteriores*, así como verde para el pasto y amarillo para senderos simulando un bosque. Para los *interiores se utilizan colores más apagados*, como gris o marrón, simulando castillos y hogares de la época.

También se dispone de un puerto, así dando la sensación de estar en una isla o pequeño continente. Esto agrega variedad al escenario como también permite variedad para la creación de NPCs que podrán ser agregados en una versión definitiva.

Por último, también se dispone de mercaderes de los que se hablará más adelante.



Jugabilidad

Al ser un juego del género RPG en dos dimensiones el jugador irrumpe desde un comienzo con un inventario vacío, que podrá ser rellenado con una gran variedad de objetos así como armas, escudos, armaduras y consumibles. También tiene la posibilidad de acumular riqueza para el sistema de comercio.

Batallas

Los combates son del tipo **combate por turnos**.

En estos, se presentan los *personajes pertenecientes al equipo actual* en la parte “footer” de la interfaz. A la izquierda de esta misma interfaz aparecen las **diversas interacciones** que podemos realizar, tales como: atacar físicamente, atacar mágicamente, utilizar un objeto consumible o huir.

Los **enemigos** se muestran entre la parte alta y media de la interfaz, de una forma que se distinga que **están enfrente**.

Hemos escogido este tipo de combate por la *accesibilidad y facilidad en el desarrollo*, priorizando otros aspectos más característicos del tipo de juego. Sin embargo, es usual encontrar este tipo de combate en RPGs; especialmente en videojuegos antiguos.

El **estilo de combate**, además, recrea una forma *oriental* de cultura; **de derecha a izquierda**; y es que, la herramienta principal usada en el desarrollo se inspira en este tipo de culturas. También por esa razón el *estilo visual*.

Para modificar la interfaz por defecto de batalla hemos utilizado el plugin HUDMaker.



Roles:

Configuración General y Curvas de Parámetro (Centro Superior)

- **Nombre:** El título de la profesión (en este caso, *Villano*).
- **Curva de EXP:** Determina cuántos puntos de experiencia necesita este rol exacto para subir de nivel.
- **Curvas de parámetro:** Gráficos que definen cómo progresan las estadísticas del personaje (PV, PM, Ataque, Defensa, etc.) desde el nivel 1 hasta el nivel máximo.
 - *Ejemplo:* En la imagen, el rol "Villano" tiene un crecimiento muy alto en **PM Máx.** (barra azul alta) y **Agilidad** (barra celeste), indicando que es un rol rápido y dependiente de magia/habilidades.

3. Habilidades a aprender (Centro Inferior)

- **¿Qué hace?:** Un listado cronológico que automatiza qué magias o técnicas desbloquea el personaje al alcanzar un nivel específico.
- **Ejemplo visual:** * En el **Nivel 1** empieza con Ataque y Destello Min.
 - Al llegar al **Nivel 2** aprende automáticamente Ceguera.
 - Al llegar al **Nivel 5** obtiene Susurro prohibido.

4. Atributos (Derecha Superior)

- **¿Qué hace?:** Define los rasgos permanentes, las pasivas innatas y las restricciones de equipamiento obligatorias de la clase.
- **Ejemplo en la imagen:** El Villano tiene configurado:
 - **Estadísticas de combate:** Tasa de acierto base de +95%, +5% de evasión y +4% de probabilidad de golpes críticos.
 - **Permisos de combate:** Puede usar el tipo de habilidad *Magia*.
 - **Permisos de equipo:** Solo se le permite equipar armas tipo *Bastón* y armaduras tipo *Armadura mágica*.

Roles	
0001 Héroe	
0002 Guerrero	
0003 Mago	
0004 Sacerdote	
0005 Villano	
0006 Esbirro	

Configuración general	
Nombre:	Villano
Curva de EXP:	[30, 20, 30, 30] ...

Curvas de parámetro			
PV Máx.	PM Máx.	Ataque	Defensa
Ataq. mág.	Defen. mág.	Agilidad	Suerte

Habilidades a aprender		
Nivel	Habilid.	Nota
Lv 1	Ataque	
Lv 1	Destello Min	
Lv 2	Ceguera	
Lv 5	Susurro prohibido	

Atributos	
Tipo	Contenido
Parámetro Esp.	Tasa del objetivo * 100%
Parámetro Ex.	Tasa de acierto + 95%
Parámetro Ex.	Tasa de evasión + 5%
Parámetro Ex.	Tasa de críticos + 4%
Añadir tipo de...	Magia
Eq. tipo de arma	Bastón
Eq. tipo de ar...	Armadura mágica

Habilidades:

Lista de Habilidades

- **¿Qué hace?:** Almacena y organiza todas las acciones especiales y comandos de combate del juego mediante un ID numérico.
- **Ejemplos:** 0001 Ataque, 0008 Curar, 0009 Fuego, 0011 Mordisco venenoso.

Configuración General e Invocación

- **¿Qué hace?:** Define la identidad de la habilidad, sus costes, restricciones de uso y cómo se ejecuta visualmente.
- **Datos clave:** Nombre, icono, coste de recursos (PM o PT), a quién afecta (ej. *1 enemigo, todos los aliados*) y la animación gráfica en combate.
- **Ejemplo en la imagen:** El "Ataque" normal no cuesta PM, afecta a un solo enemigo y tiene un 100% de probabilidad de éxito.

Mensaje y Armas requeridas

- **¿Qué hace?:** Configura el texto que aparecerá en el registro de batalla cuando se use la habilidad y restringe su uso a tipos de armas específicos.
- **Ejemplo:** Al atacar, el juego mostrará el texto: [Nombre del personaje] ¡ataca!. Podrías configurarlo para que "Ataque doble" requiera tener equipadas *Espadas*.

Habilidades	
0001 Ataque	
0002 Defensa	
0003 Ataque dual	
0004 Ataque doble	
0005 Ataque triple	
0006 Huida	
0007 Esperar	
0008 Curar	
0009 Fuego	
0010 Centella	
0011 Mordisco venenoso	
0012 Ceguera	
0013 Susurro prohibido	
0014 Destello Min	
0015 Martillazo potente	
0016 Purificar	
0017 Rayo iracundo	
0018 Ira de El Señor	
0019 Guardia Sagrada	
0020 Fé ferviente	
0021 Palabra sanadora	

Configuración general

Nombre: Icono:  76

Descripción:

Tipo de habilidad: Coste de PM: Coste de PT:

Afecta a: Ocasión:

Invocación

Velocidad: Éxito: Repetición: PT ganado:

Tipo de golpe: Animación:

Mensaje

(Usuario)

Armas requeridas

Tipo de arma 1: Tipo de arma 2:

Daño

Tipo: Elemento:

Fórmula:

Variación: Golpes críticos:

Efectos

Tipo	Contenido
Añadir estado	Ataque normal 100%

Notas

La habilidad #1 se utilizará al seleccionar el comando Atacar.

Daño y Efectos

- **¿Qué hace?:** Determina el impacto matemático de la habilidad (curación o daño), el elemento elemental y los estados alterados que puede provocar.
- **Fórmula matemática:** Utiliza variables del atacante (a) y del defensor (b). En la imagen, la fórmula es $a.atk * 4 - b.def * 2$ (cuatro veces el ataque del usuario menos dos veces la defensa del rival).
- **Ejemplos de efectos:** Añadir el estado "Veneno" con un 100% de probabilidad o recuperar un 20% de PV.

Notas

- **¿Qué hace?:** Un espacio en texto plano usado por el desarrollador para apuntes personales o, muy habitualmente, para introducir códigos (*notetags*) que activan funciones especiales de plugins externos.

Buffs y debuffs

Se han añadido una **gran cantidad de efectos**; alrededor de 10; aplicables tanto a *personajes pertenecientes al equipo principal como a los enemigos*, estos efectos disponen de **mensajes personalizados** y, cómo es evidente, *modificaciones en las estadísticas del personaje afectado*. También se han *preservado efectos por defecto* de RPG Maker.

Estados		Configuración general		Atributos
0001 Muerte		Nombre:	Icono:	Tipo
0002 Defensa		Odio comprimido	21 	Contenido
0003 Inmortal		Restricción:	Prioridad:	Parámetro
0004 Veneno		Atacar a cualquiera	50	Ataque * 110%
0005 Ceguera		[VL] Movimiento:	[VL] Animación:	
0006 Mudez		Normal	Nada	
0007 Locura		Condiciones de eliminación		
0008 Confusión		☐ Al final de la batalla ☐ Por otra restricción		
0009 Enamoramiento		Eliminación automática: Nada		
0010 Sueño		Duración en turnos: ~		
0011 Gula		☐ Remover por daño		
0012 Euforia		☐ Remover al caminar pasos		
0013 Petrificación		Mensajes		
0014 Asustado		Si un actor contrae el estado:		
0015 Soberbia		(Nom. de objetivo) odia		
0016 Taquicardia		Si un enemigo contrae el estado:		
0017 Poder de la amistad		(Nom. de objetivo) odia		
0018 Odio comprimido		Si el estado persiste:		
0019 Osadía		(Nom. de objetivo)		
0020 Ebriedad total		Si el estado es removido:		
0021 Ebriedad parcial		(Nom. de objetivo) vuelve su cordura		
0022 Subidón de ataque				
0023 Subidón de defensa				
Cambiar el máximo...				Notas

Configuración General

- **¿Qué hace?:** Define la identidad visual del estado y cómo altera el comportamiento básico del personaje afectado.
- **Restricción:** Determina si el personaje pierde el control. En el ejemplo (Odio comprimido), la restricción está configurada en "**Atacar a cualquiera**" (una especie de estado de furia o confusión).
- **Prioridad:** El orden de importancia (50). Sirve para decidir qué icono se muestra en pantalla si el personaje tiene varios estados a la vez.

Condiciones de Eliminación

- **¿Qué hace?:** Define cómo se cura o desaparece el estado de forma automática.
- **Opciones:** Puede eliminarse al terminar el combate, tras pasar un número determinado de turnos, al recibir un golpe físico (*Remover por daño*) o tras caminar cierta cantidad de pasos en el mapa.

Mensajes

- **¿Qué hace?:** Configura los textos que aparecerán en el registro de batalla cuando el estado se aplica, persiste o se cura.
- **Ejemplo visual:** * Al contraerlo: [Nombre] odia.
 - Al curarse: [Nombre] vuelve su cordura.

Atributos

- **¿Qué hace?:** Modifica las estadísticas, resistencias o elementos del personaje mientras el estado esté activo.
- **Ejemplo en la imagen:** Tiene un multiplicador de parámetro: **Ataque * 110%** (el estado aumenta la fuerza del afectado un 10%).

2. Armas

Representa el equipo ofensivo que los personajes pueden emplear para aumentar su poder de ataque o añadir propiedades elementales a sus golpes físicos.

- **Cambios de parámetro:** Su función principal es alterar las estadísticas del portador cuando está equipada.
- **Atributos especiales:** Permite asignar un elemento a los ataques del jugador (físico, fuego, etc.) o modificar estados secundarios como la tasa de acierto crítico.
- **Ejemplos:**
 - 0001 Espada: Cuesta 500 monedas y otorga **Ataque: +10**, con tipo de ataque elemental *Físico*.
 - 0005 Martillo oxidado o 0006 Bastón astillado.

Armas

0001 Espada
0002 Hacha
0003 Vara
0004 Arco
0005 Martillo oxidado
0006 Baston astillado

Cambiar el máximo...

Configuración general

Nombre:

Martillo oxidado

Icono:

109

Descripción:

Esta muy desgastado. . .

Tipo de arma:

Martillo

Precio:

120

Animación:

0001 Golpe - Físico

Cambios de parámetro

Ataque:	Defensa:	Ataq. mág.:	Defen. mág.:
15	0	0	0
Agilidad:	Suerte:	PV Máx.:	PM Máx.:
-5	0	0	0

Atributos

Tipo	Contenido
Ataque elemental	Físico
Parámetro Ex.	Tasa de acierto + 0%

Notas

Aceptar

Cancelar

Aplicar

3. Armaduras

Engloba todo el equipamiento defensivo (escudos, cascos, ropa, anillos) destinado a proteger a los héroes mitigando el daño recibido.

- **Clasificación del equipo:** Permite categorizar la pieza (por ejemplo, tipo *Escudo* o tipo *Cuerpo*) para limitar qué personajes pueden equipársela según su profesión.
- **Cambios de parámetro:** Normalmente se enfoca en mejorar estadísticas de supervivencia como la Defensa General, Defensa Mágica o los PV Máximos.
- **Ejemplos:**
 - 0001 Escudo: Cuesta 300 monedas, otorga **Defensa: +10** y añade un atributo de *Tasa de evasión +0%*.
 - 0005 Túnica polvorienta o 0006 Armadura de chatarra.

[illegible]

Comercio

El sistema de comercio aparece en el juego **nada más terminar el combate con Severino**, quien protagoniza el juego y controlamos acto seguido tras su derrota. Se nos presenta una tienda de campaña a la que al acceder se encuentra un **mercader**. Tenemos *varias opciones* en cuanto a la interacción con él. Podemos *hablar, atacar, robar, comprar o vender*; estas dos últimas se presentan combinadas. Y más adelante **en el poblado nos encontramos con más NPCs como él**.

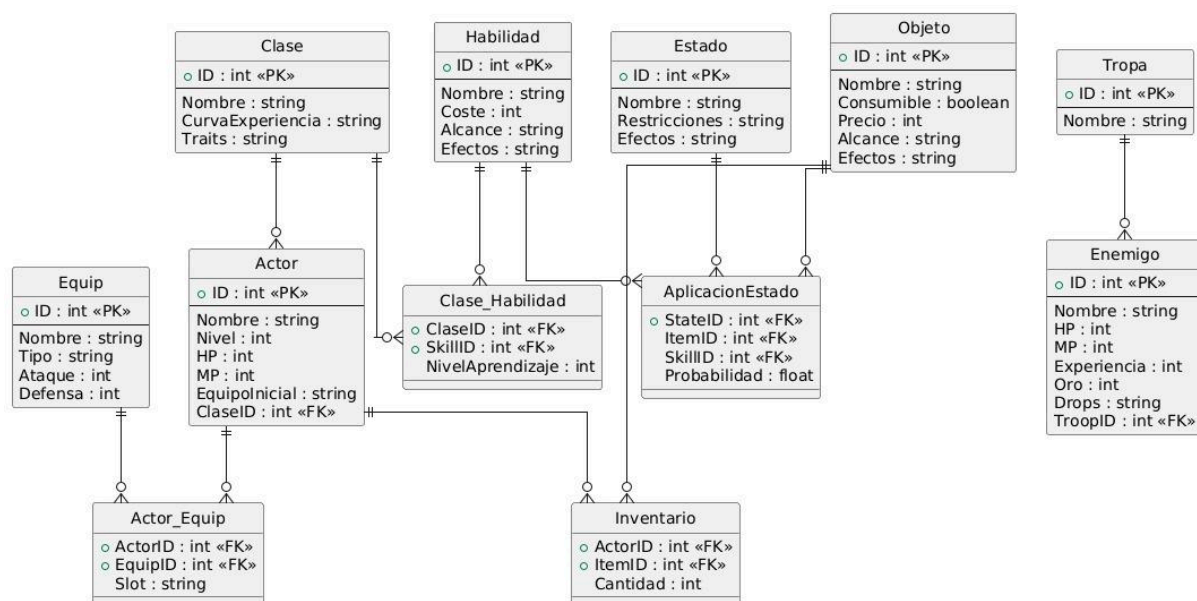


Temporalización

En esta sección aprovechamos para desarrollar cada entrega que se hizo en el seguimiento del trabajo. Separado en cinco fases. Todas con una duración aproximada de treinta horas entre los tres alumnos. Cada una de estas fases es una división orientativa para mostrar cómo va el desarrollo de nuestro juego:

Fase primera

Con entrega para el 10/03/2026 el objetivo fue estructural. Se buscaba un esquema relacional que detalle el funcionamiento de la base de datos y un diseño para ciertos aspectos del juego; mockups.



Para este modelo de diseño base de datos se ha utilizado PlantUML como lenguaje y ChatGPT para generarlo.

Inventario

Objetos

Armas

Armaduras

Objetos Clave.



Atacar

Habilidades

Bloquear

Objetos

PV: 20/20

PM: 100/100

Estados: ...





Fase Segunda: Sistema de Combate y Dinámicas de Batalla

Fecha de entrega: 24/03/2026

Esta fase se centró en la conceptualización teórica y el posterior diseño técnico del núcleo jugable (*core loop*) en lo referente a los enfrentamientos. El objetivo principal fue establecer las reglas matemáticas y el ritmo del juego en su faceta interactiva más compleja.

Desarrollo y Tareas Clave

- **Diseño del Sistema de Combate:** Definición del formato (por turnos clásico, sistema de tiempo activo/ATB o combate táctico).
- **Modelado Matemático:** Creación y balanceo de las curvas de daño, fórmulas de mitigación por defensa y cálculo de estados alterados (probabilidades de crítico, evasión y aciertos).
- **Esquema de Roles y Habilidades:** Estructuración de las sinergias entre los diferentes roles (*héroes, magos, villanos*) y cómo sus habilidades interactúan entre sí durante el flujo de la batalla.

Fase Tercera: Implementación de Infraestructura y Contenido Base

Fecha de entrega: 14/04/2026

Una vez asentadas las bases teóricas del combate, esta fase exigió dar el salto al motor de desarrollo para asentar los cimientos técnicos, visuales y lógicos del proyecto. Se tradujo el diseño de papel a datos reales y funcionales.

Desarrollo y Tareas Clave

- **Arquitectura de la Base de Datos:** Carga, estructuración y tipado en el motor de todos los elementos del juego (tablas de actores, habilidades balanceadas en la Fase 2, inventario de objetos consumibles, armas y armaduras).
- **Construcción de Escenarios Iniciales:** Bloqueo (*grayboxing*) y diseño visual de los mapas y niveles que servirán como punto de partida para la narrativa y las pruebas de juego.
- **Inyección de Scripts Básicos:** Programación o configuración de los *plugins* e hilos lógicos esenciales (controladores de cámara, mecánicas de movimiento personalizadas, gestores de interfaz de usuario) necesarios para que el prototipo sea jugable.

Fase Cuarta: Persistencia de Datos y Atmósfera Sonora

Fecha de entrega: 28/04/2026

Con el contenido base ya interactivo, el foco de esta etapa se dirigió hacia la robustez técnica del software y la inmersión del usuario a través del apartado auditivo.

Desarrollo y Tareas Clave

- **Sistema de Persistencia (Guardado Estable):** Programación y testeo del ciclo de guardado y carga de datos. Se aseguró la correcta serialización de variables complejas (posición del jugador, estado de las misiones, inventario actual y estadísticas de los roles) evitando la corrupción de archivos.
- **Diseño de Sonido (SFX):** Integración de efectos de sonido para dar *feedback* inmediato a las acciones del jugador (pasos, confirmación de menús, golpes físicos y efectos mágicos en combate).
- **Curaduría e Implementación Musical (BGM):** Selección e integración de las pistas musicales de fondo, asegurando transiciones fluidas entre la exploración de escenarios y el inicio de los encuentros de batalla.

Fase Quinta y Final: Documentación, Cierre y Presentación

Fecha de entrega: 12/05/2026

La fase de cierre se enfocó en el control de calidad final, la justificación técnica del proyecto y la preparación del material de cara a la defensa o entrega formal.

Desarrollo y Tareas Clave

- **Redacción de la Documentación Técnica:** Elaboración del Documento de Diseño de Videojuego (GDD), manuales de usuario, diagramas de flujo de los scripts y memoria técnica del proyecto.
- **Pruebas de Integración Final (QA):** Fase intensiva de *playtesting* para la detección y corrección de *bugs* de última hora, asegurando que el producto final sea completamente estable.
- **Preparación del Material de Presentación:** Creación de diapositivas, capturas de demostración y empaquetado (*build* ejecutable) del proyecto final listo para su exposición.